

Analysis and Comparison of UART, SPI and I2C

Jinrong Chen^{†,*}XJTLU University
Suzhou, 215123, ChinaShengqiu Huang[†]Shanghai United International School, Wanyuan Campus
Shanghai, 200050, China

Jinrong.Chen20@student.xjtlu.edu.cn

*Corresponding author

[†]These authors contribute equally.

Abstract—Nowadays there are many hardware communications protocols to choose from, UART, SPI, I2C three protocols are the most representative, they are widely used. They have different characteristics, and how to choose the right choice is a problem that has been bothering people for a long time. In this article, the composition of the three protocols, how they work, and their advantages or disadvantages are described separately. And through the comparison of their transmission distance, it will be found that today's SPI protocol can transmit a longer distance, in the comparison of transmission rates SPI protocol has a faster rate but many limited conditions, UART has a more stable performance, SPI has a variety of transmission modes to choose from. In terms of transmission power consumption, UART has a huge advantage, outperforming the other two protocols. Through the article, we hope to provide a new way of thinking from the purpose required for the design. This leads to a new perspective on protocol selection

Keywords--UART, SPI, I2C

I. INTRODUCTION

Various communication protocols are available nowadays, each protocol has its own advantages and disadvantages, and the choice of hardware transmission protocol is also different in different situations. The topic to be covered in this post is how to pick a suitable protocol. The following article discusses a few of the most well-liked communication protocols now in use and examines numerous protocols' properties to determine which are preferable and superior. UART is a widely used hardware communication protocol that requires only two data lines to complete bidirectional data transmission and does not require clock signals for synchronous operation. It transfers data in packets, each with a variable size of data. A check digit can be included in the package to guarantee security and integrity during data transmission. Synchronization between the transmitter and receiver is achieved by bit rate, which allows two devices operating at different frequencies to communicate. The SPI protocol is a very fast transmission protocol that allows the master to communicate with the slave at a higher frequency. The SPI protocol allows only one master to exist, but multiple slaves can exist. The master is responsible for generating a clock signal that is used to synchronize with its slave. The SPI saves pins by occupying only 4 wires in the chip, which also benefits saving PCB layout space. The I²C protocol is a two-wire serial bus that requires only two bidirectional lines, a serial data line, and a serial clock line. The bus interface is formed within the chip, optimizing space and cost. It supports multi-master and multi-slave devices, but only one host is allowed at a time. The number of I2C protocols connected to the bus is limited only by the maximum capacitance of the bus. It is

characterized by low power consumption and strong anti-interference ability. Through the working principle and characteristics of the above protocols, we get a better solution in the specific working environment or the general working environment and provide new solutions for the application of the protocol in different scenarios, in the pursuit of low-power environment solutions or the choice of long-distance transmission, or the pursuit of excellent choices under high-speed transmission. The entire article aims to offer fresh suggestions for protocol collocation and selection.

II. UART

Universal Asynchronous Receiver/Transmitter is an Asynchronous serial communication protocol. It is one of the most widely used hardware-to-hardware communication protocols. Nowadays UART has been used in many applications such as various types of environmental sensors, wireless communication modules, and so on [1].

A. Working principle of UART

UART is a serial communication protocol, that differs from parallel communication, transmissions in that the transmitted data does not reach the receiver at the same time [2]. It is transmitted bit by bit to the receiver via a data line. This means that its transfer rate may be slower than the transfer rate of the parallel transfer protocol. The transmitter's TX pin is directly connected with the receiver's RX pin. Data travels to and from a UART in parallel to the control device, such as microcontrollers and microprocessors. When sending on the TX pin, the transmitter's UART needs to translate parallel information into serial and transmit it to the receiver. The receiver receives this data and translates it back to parallel, then communicate with its controller. Another feature that distinguishes UART from other communication protocols is that the transmitter and receiver are not synchronized through the clock signal. Instead, the transmitter and receiver synchronize with each other at the same baud rate. The baud rate is expressed in bits per second or bps, it is usually 9600 or 115200.

B. Structure of the UART

UART transmitted data is organized into packets. Each packet contains 1 start bit, 5 to 9 data bits, a parity bit (ability to choose whether to use or not), 1 or 2 or 1/2 stop bits.

START BIT: The UART protocol is in a high-level state when the data line is idle. When data transfer starts, the transmitter will change the level of the transmission line from high to low for one clock. It is added before the actual data. The receiver detects this change from high to low on the data line and starts receiving the actual data.

DATA BITS: The data bits are the actual data that is

transmitted between transmitter and receiver. The length of data can be anywhere between 5 to 9 bits. In particular, 9-bit data is the resulting length without the use of parity bits. Most of the time the maximum data length is 8 bits.

PARITY BIT: The parity bit can help the receiver check if any data has changed during the transmission. Bits can be changed in kinds of error, such as receiving overflow lost data, the reason it has received lots of data but it cannot process at once. Unfinished UART transmission results in data loss, use this function to enter hibernation after sending, power off the receiving device, and so on. After the receiver reads the data, the number of digits is counted as 1 and determines whether the total is odd or even if the parity bit is 0, the number of 1s in the packet is even, and if the parity bit is 1, the number of 1s in the packet is odd.

END BITS: As the name suggests, it is marking the end of the data packet. Typically, the stop bit occupies 1 or 2 clocks, but it is also possible to set 1/2 clocks. When the stop bit ends, the data line is idle and its level returns to the high position.

C. Features of UART

Unlike other hardware communication protocols, UART only requires two data lines for full-duplex data transmission, such as the SPI protocol, which requires at least four data lines to be connected to each other. UART also does not require a clock or other time signal for communication between two devices. Because of the presence of a parity bit, the basic accuracy of the transmitted data is ensured when used. UART has a wide range of hardware applications and a variety of use scenarios. The UART protocol also has excellent power performance, enabling an advance in the development of green communication technology and is proven in Artix-7[3].

As for its disadvantages, there are mainly the following points: The maximum package size transmitted is limited to 9 bits. Compared to parallel transmissions, UART has a lower transmission rate. The UART protocol is not a bus communication, it cannot communicate with multiple devices at the same time, at a time can only be one-to-one communication. Unlike the SPI protocol, a host can have multiple slaves, the I2C protocol can allow multiple hosts to exist with multiple slaves. The difference in baud rate between two UART protocol devices that communicate with each other should not exceed 10%.

III. SPI

The serial Peripheral Interface is known as SPI. It is a full-duplex, high-speed synchronous communication bus that only uses four pins on the chip, saving the chip's pins, freeing up space, and making the PCB layout easier. More and more chips, including AT91RM9200, are integrating this communication protocol because of how simple it is to use. The working principle is shown in Figure 1 below.

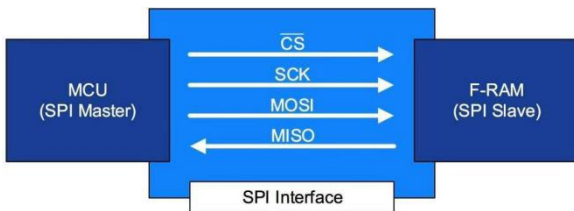


Figure 1. Master-slave diagram

In master-slave mode, SPI operates. The slave is typically an EPROM, Flash, AD/DA, audio and video processing chip, or other devices, while the host is typically a programmable controller such as an FPGA, MCU, or DSP. The SCLK, CS, MOSI, and MISO lines make up the majority of them. SCK, SS, SDI, and SDO are examples of names that all have the same meaning. When there are multiple slave devices, the CS is used to select the slave device to be controlled.

A. Working principle of SPI

In general, the working mode of the machine parts is fixed, and the host machine needs to adopt the same working mode. The Controller generally refers to the control registers in the SPI device, which can be configured to set the transmission mode of the SPI bus, so that both sides can normally "communicate".

When the SPI is idle, the clock signal's polarity—CPOL—shows whether it is high or low. When the device is not in use, the clock signal at the SCK tube's foot is high if CPOL is set to 1. When CPOL is set to 0, the opposite is accurate. SCK is zero when it is not in use, as shown by CPOL=0. When SCK is in use, CPOL=1 indicates that it is 1.

CPHA: clock phase, which specifies whether the SPI device starts sampling data when the clock signal on the SCK pin transitions to a rising edge or a falling edge. If CPHA is set to 1, the clock signal's rising and falling edges cause the SPI device to sample data and transfer data, respectively. When CPHA is set to zero, the opposite is accurate. At the first edge of SCK, CPHA=0 denotes that the input and output data are legitimate. The input and output data are valid at the second edge of SCK when CPHA=1[4]. Of the four modes, modes 0 and 3 are the most widely used, and most SPI devices support both modes.

B. Structure of the SPI

By default, when we discuss SPI, we are referring to the conventional 4-wire Motorola SPI protocol, which has four data lines total: SCLK, MOSI, MISO, and CS. The typical 4-wire system has the benefit of enabling full duplex data transfer. An SPI interface can have only one host, but it can have one or more slaves. One CS is required when there is just one host and one slave device, however, many slave devices require several CS. MOSI and MISO are data lines. MOSI sends data from the master to the slave, and MISO sends data from the slave to the master [5]. The beginning of every data line: clock signal, or SCLK. The host output, slave input, host data MISO, host input, slave output, CS, slave device selection, and active low level are all components of the SPI rate, which is determined by the clock frequency. The connection schematic was shown in figure 2 below.

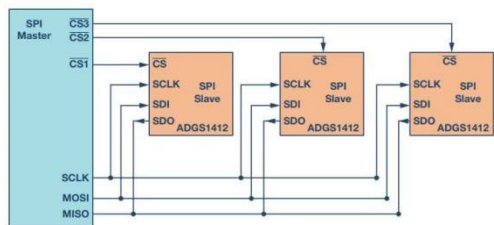


Figure 2. Working connection diagram

There are two primary varieties of 3-wire SPI, depending on the various application scenarios:

For simcom communication, some peripherals have only

three SPI buses, one CS, one CLK, and one MOSI/MISO, which means that the input and output of data are the same data line. The SCLK, SDIO, and CS wires are the only ones present. A bidirectional port for half-duplex communication is the SDIO. For instance, ADI's numerous ADC chips enable bidirectional transmission. Bidirectional ports should be operated by FPGAs with high resistance state Z set when utilized as input.

C. Additional information

There are several special characteristics shown through the SPI bus, not only limit to its high-frequency communication speed but also the daisy chain connection type can be shown that SPI is a superior communication protocol.

1) Different connection structure

SPI has two types of connections. The traditional independent slave setup is the first. Each slave requires their own CS line. The matching CS signal line is brought down while the other CS signal lines are kept high when the host wants to talk to a certain slave. The MISO pins of the unselected slave machine must be set up with high-resistance

output since they are on the same signal line as the slave machine's MISO pins.

Daisy chain configuration is SPI's second connecting option. Daisy chaining is the process of serially transmitting signal wires from one device to the next until the data reaches the destination device [6].

The primary drawback of the Daisy chain is that it will disconnect the slave machine that has a lower priority than the device if the slave machine has a single point of failure. The priority for service will be lower for the slave machine that is further away from the host machine. If necessary, configure the priority going forward and the slave line detector [6].

To avoid a single point of failure causing the entire link to fail, respond quickly if a slave server times out.

The SPI shift register is fully utilized in daisy-chain mode when each slave copies input data to output in the subsequent clock cycle. The difference comparison diagram is shown in Figure 3.

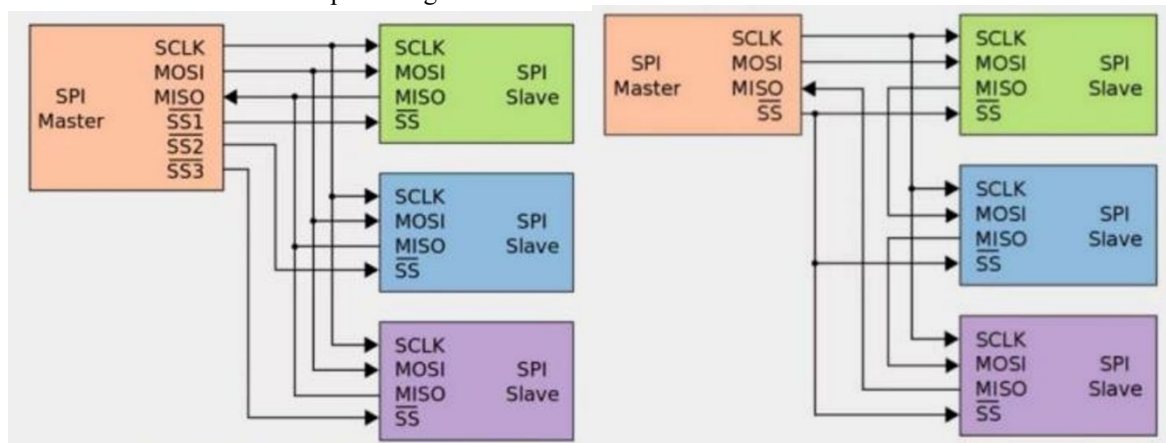


Figure 3. Link comparison diagram

The primary drawback of Daisy chain is that it will disconnect the slave machine that has a lower priority than the device if the slave machine has a single point of failure. The priority for service will be lower for the slave machine that is further away from the host machine. If necessary, configure the priority going forward and the slave line detector.

To avoid a single point of failure causing the entire link to fail, respond quickly if a slave server times out.

The SPI shift register is fully utilized in daisy-chain mode when each slave copies input data to output in the subsequent clock cycle.

2) Transmission rate

In contrast to I2C's normal mode of 100K, fast mode of 400K, and high-speed mode of 3.4m, the SPI protocol does not have a set rate. Instead, the maximum SPI clock frequency, the CPU's capacity to interpret SPI data, and the driving capacity of the output end all play major roles in determining the SPI transmission rate (maximum signal transmission rate should be followed by the PCB layout).

IV. I2C

The full name of the I2C bus can be called IIC or Inter-Integrated Circuit. The number two in the phrase would likely be square, not two. The I2C is a synchronous, multi-controller and targets serial communication bus, developed by Philips Semiconductor in 1982. The most significant point of this serial communication bus is that it can connect with multiple devices with two lines, and the device can have the chance to change its characteristic with master and slave [7]. Also, there are different communication modes that can be chosen by developers. I2C is used to connect devices like microcontrollers, EEPROMs, I/O interfaces, and other peripheral devices in an embedded system [8]. For example, air pressure sensor connected between Arduino and the sensor would be a perfect application for the I2C bus. The ADC and DAC chip would convert the analog data to a digital i2c signal to keep the number constant and accurate.

A. Working principle of I2C

I2C uses two wires, SCL and SDA, also known as Serial Clock Line and Serial Data Line, to connect with dozens of devices. Each device owns an addressing code with 7 bits. From i2c-bus.org, it suggested that the bite width with 7 can theoretically allow 128 I2C addresses, but for certain reasons,

there are several addresses with special purposes, which are “0000000 0, 0000000 1, 0000001 X, 0000010 X, 0000011 X, 00001XX X, 11110XX X, 11111XX X”, has been noted as binary, 10-bit addresses [7].

The i2c simulation and concepts are based on the square wave oscillation, in which electromagnetic or other interference has not been considered. By using capacitors on both SCL and SDA wire could reduce and filter the noise of the wave. However, the character of the capacitor and the charging period would affect by the size of the capacitor. It is extremely important to consider what size of the capacitor to use since the clock and data line should reach the logical high voltage, or the message wouldn't be recognized [8]. From Texas Instruments' suggestion, the capacitor used for the I2C bus of each device should follow the 400 pF for both Standard Mode and Fast Mode and specifies 550 pF for Fast Mode Plus [9]. The following figure 4 shows the waveform.

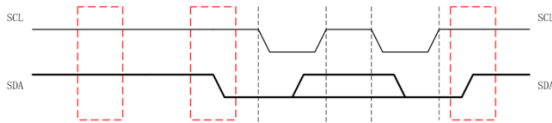


Figure 4. Operation waveform diagram

B. Structure of the I2C

As the I2C bus is resting, both SCL and SDA wires are keeping HIGH. When the device is planning to transfer data, SDA would switch to LOW and start transferring data through the SDA wire, and the clock wire would provide the clock message to each of the devices. When the device is planning to stop transferring data, the SDA would switch from LOW to HIGH while the SCL is in HIGH condition.

For writing data, the device first sends the stop signal to the bus, reduces the condition of failing to write; then sends the rest signal, the start signal. Send the address of 7 bits, and the slave device would send back a feedback signal. Then start transferring data with 8 bits, getting back a feedback signal from the slave as a period. For ending, send a stop signal to the I2C bus, and release the bus in an idle state.

C. Flaws of I2C

For several reasons, there are some disadvantages that I2C is not suggested to be applied in long-distance communication. Though there are some ways that can solve the bit flip issues, the parasitic capacitance from the PCB or wire cannot be solved as a physical defect. If the material of the wire is inferior, the fluctuation of the parasitic capacitance and the resistance of the wire will further affect the stability of the i2c bus. If the resistance is too large, the master cannot parse the data of the slaves, which will lead to a series of unreliable problems.

V. COMPARISON

The previous chapters describe the working principles and features of the three protocols, and in this chapter they will be compared in a single direction. In terms of transmission distance, each of the three protocols has its own limited length, and the transmission distance of the UART protocol is 15m in general. However, in the design based on RS-485, it is possible to achieve a long-distance transmission of 100m. The SPI protocol is typically a short-range transmission protocol that can achieve transmission distances of more than 1200m in a specially designed environment. I²C is a short-range transmission protocol whose transmission

distance is about 200mm to 300mm without the use of repeaters. As can be seen in the above transmission characteristics, the SPI protocol can be used under the requirements of the application environment that requires long-distance transmission. The UART protocol operates at different rates under different standards at different transfer rates, with a transfer rate of 20Kbps under RS-232 standards. Under the RS-422 standard, its transmission rate can reach 10Mbps. And the longer the transmission distance, the smaller the maximum transmission rate. The transfer rate of the I²C protocol has different maximum transfer rates in different modes, and its transfer rate is up to 100Kbps in standard mode. In ultra-fast mode it has a maximum transfer rate of 5Mbps. [10] The transmission rate of the SPI protocol can reach up to 50Mbps, but its maximum transmission rate is subject to three conditions: 1. System clock frequency 2. CPU processing SPI data capability 3. Maximum signal transmission rate allowed by the PCB. If the design requires high-speed transmission, the SPI protocol's high rate is an option. If the design requires simple connections and a streamlined layout, I2C can do it. If the design is designed to save energy, then UART is the best choice, it consumes much less power than the other two protocols at 0.0135W.[11]

VI. CONCLUSION

In this experiment, after studying the working principle, format description and personality characteristics of UART, we found that: The transmission rate of UART may be lower than some parallel protocols, and the transmitter and receiver are synchronized with each other without any medium. And use packet form in transmission. The most important thing is that UART data transmission only requires two data lines, and it is a full duplex system, although it can only be one-to-one, it can have multiple slave machines to achieve one-to-many.

Then, we analyse all aspects of the SPI protocol and conclude that SPI as a full-duplex and high-speed communication line can simplify the layout and the host can change the controller by itself. The fourth line is a full duplex, while the third line is only half a duplex. It can be configured to set the main transmission mode, and the rate is constantly changing. Chrysanthemum link is undoubtedly a very good way of application, but the disadvantage is easy to cause the failure of the point.

Finally, we reviewed the relevant literature on I2C and made a self-perception. I2C can connect multiple Settings with two lines, its characteristics can be changed, and the wave noise can be reduced to a certain extent. Finally, it has high requirements on wire material, because it will affect the stability of the bus, and too much resistance will affect the analysis of server data.

REFERENCES

- [1] Kashyap, B., & Ravi, V. (2020). Universal Verification Methodology Based Verification of UART Protocol. *Journal of Physics:Conference Series*, 1716(1), 012040.
- [2] Sharma, R., Pandey, B., Jha, V., Saurabh, S., & Dabas, S. (2018). Input-Output Standard-Based Energy Efficient UART Design on 90 nm FPGA. *System and Architecture*, 139–150.
- [3] HariPriya, D., Kumar, K., Shrivastava, A., Al-Khafaji, H. M. R., Moyal, V., & Singh, S. K. (2022). Energy-Efficient UART Design on FPGA Using Dynamic Voltage Scaling for Green Communication in Industrial Sector. *Wireless Communications and Mobile Computing*, 2022, 1–9.
- [4] Slee, V. N. (1976). 27. CPHA Patient Care Data System for Long-Term

- Care Facilities. *Medical Care*, 14(Supplement), 209–212.
- [5] Tan Xian-jun & Jin Yan. (2011). I/O Port Software Simulation of three-wire SPI communication timing. *Journal of Zhejiang University of Technology* (05), 571-573+585.
 - [6] Whipple, T. (2019). Automated Daisy Chain Generation and Verification. Additional Conferences (Device Packaging, HiTEC, HiTEN, and CICMT), 2019(DPC), 001005–001027.
 - [7] Leens, F. (2009). An introduction to I2C and SPI protocols. *IEEE Instrumentation & Measurement Magazine*, 12(1), 8–13. <https://doi.org/10.1109/mim.2009.4762946>
 - [8] Chyre Tuang Sng, T. (2010). Galvanic Isolation Strengthens I2C Communications. *Electronic Design*, 58(5), 53–57.
 - [9] Houde, F. (2018). Why, When, and How to use I2C Buffers. Texas Instruments. <https://www.ti.com/lit/an/scpa054/scpa054.pdf>
 - [10] Semiconductors, P. (2003). I2c manual. 2003-01. [2015-12-01] <https://www.nxp.com/docs/en/application-note/AN10216.pdf>
 - [11] Murali, A., Kakarla, H. K., & Anitha Priyadarshini, G. M. (2021). Improved design debugging architecture using low power serial communication protocols for signal processing applications. *International Journal of Speech Technology*, 24(2), 291302.